Multi-Tenancy en PostgreSQL

Más allá del tenant_id

Sobre mí

Eduardo Ismael

Senior Software developer

@plusplus

Desarrollador Backend con Python y Ruby

12 años en la industria



Sobre mí

Eduardo Ismael

Senior Software developer

@plusplus

Desarrollador Backend con Python y Ruby

12 años en la industria

Se hacer Joins 😄



Un Saas 🪴

Cada company necesita manejar su propia data

Una misma aplicación para múltiples clientes



Acme

Tenant A



Stark

Tenant B



Wayne

Tenant C

Podemos compartir una misma base de código, infraestructura, observabilidad etc...
Pero NO podemos compartir datos.

Es importante distinguir dos conceptos

Multi-tenancy

Es el requisito de arquitectura.

Schemas de PostgreSQL

Son un mecanismo posible para implementar ese requisito.

Tres posibles opciones

01

Shared tables

`projects.tenant_id`

02

Schema per tenant

`acme.projects`

03

Database per tenant

`acme_db`

Opción 1

Shared tables.

Una columna para conocer el owner.

```sql
CREATE TABLE projects (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT NOT NULL,
  name TEXT NOT NULL
);
```

```
id  tenant_id  name
--  _________  ________________
1   1          Acme Website
2   1          Acme API
3   2          Stark Dashboard
```

```sql
SELECT *
FROM projects
WHERE tenant_id = 1 -- Acme;
```

Tenant_id responde a la pregunta **“¿A quién pertenecen estos datos?”**

Desventajas

Un mal filtro puede convertirse en un data leak

Esperado

```sql
SELECT *
FROM projects
WHERE tenant_id = 1;
```

Bug

```sql
SELECT *
FROM projects;
```

Por eso los diseños de shared tables suelen usar **defense in depth**.

Application scoping → RLS → constraints → permisos

Protección en PostgreSQL

Row Level Security (RLS)

```sql
ALTER TABLE projects
ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_policy
ON projects
USING (
  tenant_id =
  current_setting('app.current_tenant')::BIGINT
);
```

App

Establece el tenant context

→

PostgreSQL

Filtra las rows permitidas

La database puede hacer cumplir el boundary aunque el query esté incompleto.

Opción 2

Schema per tenant

Instancia compartida

PostgreSQL

→

Una misma Database con N número de schemas

Public	**Acme**	**Stark**	**Wayne**	**...**
`tenants`	`projects`	`projects`	`projects`	
`domains`	`users`	`users`	`users`	
	`artifacts`	`artifacts`	`artifacts`	

metadata compartida

La database sigue compartida. Cada tenant tiene su propio schema y su propia información.

¿Qué son los schema?

```sql
CREATE SCHEMA acme;
CREATE SCHEMA stark;
CREATE SCHEMA wayne;

CREATE TABLE acme.projects (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL
);

CREATE TABLE stark.projects (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL
);

CREATE TABLE wayne.projects (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL
);
```

Un **schema** es un namespace dentro de una misma base de datos. Agrupa tablas, vistas, funciones y otros objetos bajo un nombre.

`acme.projects`, `stark.projects` y `wayne.projects` pueden existir juntas en la misma base de datos sin compartir filas ni datos.

En este modelo, `tenant_id` suele ser innecesario dentro de las tablas del tenant.

El mismo nombre, otra relation

Aunque ambas se llamen `users`, `acme.users` y `stark.users` son relations distintas.

Cada una tiene lo suyo

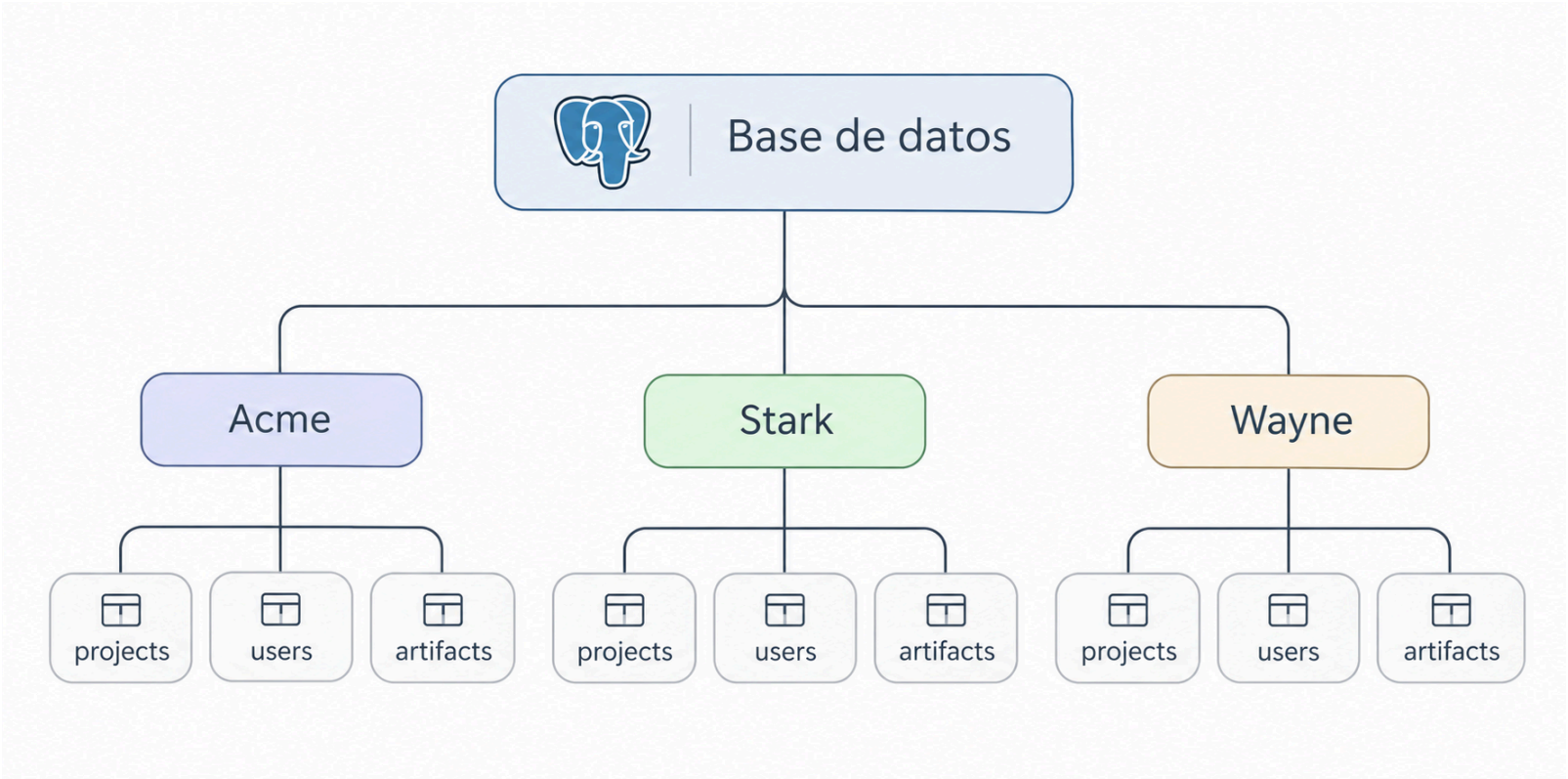
Storage físico y rows
Indexes y statistics
Primary keys y unique constraints
Foreign keys
Locks a nivel de tabla
Autovacuum y estado de mantenimiento

Qué es un schema

No es otra database ni un storage engine. Es, ante todo, un **namespace** y un **permission boundary**.

PostgreSQL documenta esta distinción.

Qué tenemos


Base de datos
Acme
Stark
Wayne
projects
users
artifacts
projects
users
artifacts
projects
users
artifacts

Comandos y Queries

```
live=# \dn
 List of schemas
  Name  |  Owner
--------+---------
 acme   | eduardo
 public | eduardo <-- Default
 stark  | eduardo
 wayne  | eduardo

\dt acme.*
          List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+---------
 acme   | projects | table | eduardo
(1 row)
```

```
INSERT INTO stark.projects (name)
VALUES
  ('Stark Dashboard'),
  ('Arc Reactor Monitoring'),
  ('Stark Inventory');
```

```
live=# SELECT * FROM stark.projects;
 id |          name
----+-------------------------
  1 | Stark Dashboard
  2 | Arc Reactor Monitoring
  3 | Stark Inventory
(3 rows)

live=#
```

PostgreSQL ya utiliza schemas. Si no se define uno, se utiliza `public` por defecto.

Routing

`search_path` le dice a PostgreSQL dónde buscar

```
SET search_path
TO stark, public;

INSERT INTO projects (name)
VALUES ('Stark Satellite Uplink');

SELECT * FROM projects;
```

```
 id |          name
----+-------------------------
  1 | Stark Dashboard
  2 | Arc Reactor Monitoring
  3 | Stark Inventory
  4 | Stark Satellite Uplink
```

PostgreSQL resuelve:

```
projects
   ↓
stark.projects
```

Si no lo encuentra:

```
public.projects
```

django-tenants

Un request, un schema

```
stark.example.com
        ↓
Tenant middleware
        ↓
resolve tenant = stark
        ↓
search_path → stark, -public-
        ↓
Project.objects.all()
        ↓
stark.projects
```

La librería elige el schema. El query del ORM no cambia.

Enforcement

Los schemas pueden tener permisos

```
GRANT USAGE
ON SCHEMA acme
TO acme_role;

GRANT SELECT, INSERT,
UPDATE, DELETE
ON ALL TABLES
IN SCHEMA acme
TO acme_role;
```

Dos ideas distintas

search_path

¿Dónde debe buscar PostgreSQL?

GRANT / REVOKE

¿A qué puede acceder realmente este role?

Application roles

Least privilege Principel

app_login 🔑	→	**acme_role**	**stark_role**	**wayne_role**
PostgreSQL User		acme.*	stark.*	wayne.*

La connection puede servir a muchos tenants,
pero cada request opera con los permisos de **un solo tenant**.

El login tiene la capacidad de asumir varios roles.
El **current role** limita los privilegios activos durante el request.

django-tenants

Un request, un role

```
stark.example.com
        ↓
Tenant middleware
        ↓
resolve tenant = stark
        ↓
SET LOCAL ROLE stark_role
        ↓
search_path → stark, public
        ↓
Project.objects.all()
        ↓
stark.projects
```

El schema rutea. El role limita qué puede tocar este request.

Least privilege por request

Reduciendo el scope

Request de Acme ✅

```
BEGIN;

SET LOCAL ROLE acme_role;

SET LOCAL search_path
TO acme, public;

SELECT *
FROM projects;

COMMIT;
```

Durante esta transacción:

```
acme.projects   ✅
stark.projects  ❌
wayne.projects  ❌
```

Bug accidental ❌

```
SELECT *
FROM stark.projects;
```

PostgreSQL responde:

```
ERROR: permission denied
for table projects
```

Porque el role activo es:

```
current_user = acme_role
```

Aunque `app_login` pueda asumir `stark_role` en otro request, **este request** está limitado a los permisos de `acme_role`.

Idea clave

No es aislamiento absoluto

Sí protege contra ✅

- Queries al schema equivocado
- Accesos accidentales
- Bugs en queries
- Un `search_path` incorrecto
- Mayor blast radius por request

No protege contra ❌

- La app seleccionando el role incorrecto
- Una app completamente comprometida
- `app_login` usando otro tenant role intencionalmente

El login tiene capacidad global.
El request trabaja con privilegios locales.

¿Un DB user por tenant… o un superuser?

Runtime role compartido

Buena idea cuando

Los tenants comparten database y pool.
La aplicación selecciona el schema correcto y el runtime role tiene privilegios mínimos.
Quieres un solo set de credentials para el tráfico web.

Suele ser el default

Un role por tenant

Buena idea cuando

Compliance o un contrato exigen que el DB niegue el acceso cruzado.
Debes revocar un tenant sin tocar a los demás.
El número de tenants todavía se puede operar.

Más secrets, pool más difícil

Superuser

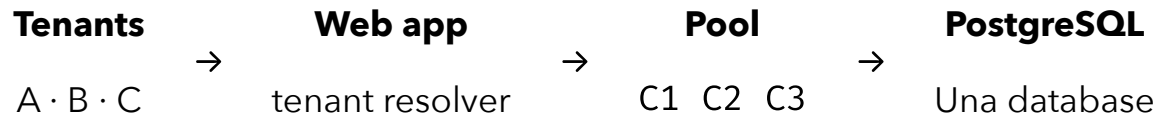
Buena idea cuando

Es una sesión humana de administración.
No es el role que atiende requests.
Las migrations usan otro role, con DDL, no este login.

Nunca para el tráfico web

Desventajas ❗

Connection pooling



Una connection no debería pertenecer de forma permanente a un tenant.

`search_path` puede ser peligroso con connections reutilizadas.

Riesgo del pooling

El estado del tenant puede filtrarse por una connection reutilizada

Riesgoso

```
SET search_path TO acme;

-- el request termina
-- la connection vuelve al pool
```

¿Qué estado hereda el siguiente request?

Idea genérica con scope de transacción

```
BEGIN;

SET LOCAL search_path
TO acme, public;

-- queries

COMMIT;
```

Las connections pertenecen al pool.
El tenant context pertenece al request o a la transacción.

Noisy neighbor

Un tenant ruidoso vuelve lenta a toda la database

Qué se comparte

CPU y I/O

Locks

Connections

`shared_buffers`

Qué ayuda

Índices y queries acotados

`statement_timeout`

Mover al tenant ruidoso a otra database

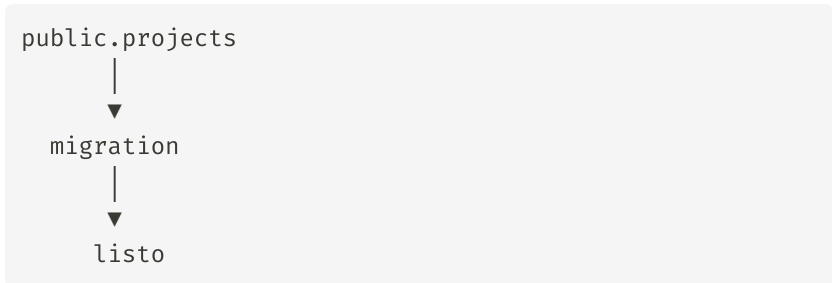
Si el CPU o el disco no alcanzan, a otra instancia

Un reporte o un import de Acme hace esperar al resto de tenants.

Schema-per-tenant aísla los datos, no los recursos.

Las migrations pueden volverse un problema

Shared table



Schema per tenant

```
acme.projects   → migrate
stark.projects  → migrate
wayne.projects  → migrate
tenant_004      → migrate
tenant_005      → migrate
...
```

Una migration lógica se convierte en **N migrations físicas**.

¿Qué pasa si el Tenant C falla?

```
Tenant A  → v15  ✅ committed
Tenant B  → v15  ✅ committed
Tenant C  → v14  ❌ migration fallida
Tenant D  → v14  ⏳ pending
Tenant E  → v14  ⏳ pending
```

Si A y B ya hicieron commit, normalmente **siguen migrados**.

El Tenant C puede hacer rollback de **su propia** migration.

Ahora producción contiene temporalmente **varias versiones de schema**.

Más observabilidad

1. Track

```
acme → v15
stark → v15
wayne → v14
```

2. Retry

Omite los tenants completados.
Reintenta solo los tenants fallidos o pending.

3. Report

Saber exactamente qué tenant falló y por qué.

Un migration runner debe ser **restartable**, no “todo o nada para siempre”.

¿Qué pasa si Acme necesita una columna custom?

```sql
ALTER TABLE acme.projects
ADD COLUMN cost_center TEXT;
```

```
Acme → projects + cost_center
Stark → projects estándar
Wayne → projects estándar
```

Técnicamente fácil.

Operativamente caro.

Las migrations divergen
Las suposiciones del ORM divergen
El testing crece
Las versiones de los tenants se vuelven más difíciles de razonar

Prefiere **JSONB** o **JSON** cuando la personalización es data, no un producto realmente distinto.

Cuándo consideraría schema-per-tenant

Schema-per-tenant es atractivo cuando…

Buen fit

Un namespace claro por tenant importa

Los permisos por schema son útiles

El número de tenants es manejable operativamente

Puedes invertir en orquestación de migrations

Cuidado cuando

El número de tenants puede volverse masivo

El schema cambia mucho por cada tenant

Las migrations son frecuentes o pesadas

Opción 3

Database per tenant

Instancia compartida

PostgreSQL

→

Una database por tenant

acme_db

```
projects
tasks
```

stark_db

```
projects
tasks
```

wayne_db

```
projects
tasks
```

El boundary es la database. Un query en `acme_db` no ve las tablas de `stark_db`.

Opción 3

Database per tenant es atractivo cuando…

Buen fit

Hay que hacer backup, restore o drop de un tenant sin tocar a los demás

Un tenant es muy grande, o tiene compliance propio

Quieres límites de recursos distintos por tenant

El número de tenants se mantiene bajo

Cuidado cuando

Cada request tiene que rutear a otra connection string

El pool se fragmenta: más connections hacia PostgreSQL

Provisioning, backups y migrations se multiplican por database

Un reporte cross-tenant ya no es un SELECT

Conclusión

El boundary que eliges es el costo que operas

01

Shared tables

El boundary vive en el **row**.

Un filtro olvidado es un data leak.
RLS es la red de seguridad.

02

Schema per tenant

El boundary vive en el **schema**.

`search_path` rutea. GRANT aísla.
Cuida el connection state y las migrations.

03

Database per tenant

El boundary vive en la **database**.

El aislamiento es el más claro.
Pagas routing, pools y operaciones por tenant.

No hay un ganador universal. Elige el boundary que tu equipo puede operar.

Muchas gracias. 